PART 3 · CHAPTER 9

MCP in Plain English

Connecting Claude to real tools — databases, GitHub, Slack, and everything else your development workflow depends on

---

Most of this book treats Claude Code as a coding assistant — something you talk to in a terminal, give a task, and watch produce files. That framing is accurate and useful, but it does not capture the full picture of what Claude Code is becoming. The Model Context Protocol changes the nature of the tool fundamentally. Instead of Claude Code operating on your codebase in isolation and asking you to bridge every gap to the outside world, MCP connects it directly to the real services your development workflow depends on: the database where your data actually lives, the GitHub repository where your issues actually are, the Slack channel where your team actually communicates. When you work with MCP-connected Claude Code, you are not describing the world to the model and hoping it understands. You are giving the model direct access to the world.

The name Model Context Protocol is accurate but not evocative. A better mental model: MCP is the plugin system for Claude Code. Just as a browser becomes genuinely useful only when extended with the tools a specific user needs, Claude Code becomes genuinely integrated into your workflow when extended with connections to the services that workflow depends on. Without MCP, Claude Code knows about your codebase and about the general state of the world as of its training cutoff. With MCP, it knows about your codebase and about the specific, real-time state of your database, your issue tracker, your deployment status, and your team's recent conversations.

The practical implications are significant. A Claude Code session connected to your PostgreSQL database can read the actual schema, query the actual data, identify slow queries with real execution plans, and suggest indexes for the actual query patterns your application uses — not for a hypothetical schema that was described in a prompt. A session connected to your GitHub repository can read open issues, understand their context, find related code, and propose fixes without you copying and pasting issue descriptions. A session connected to your monitoring service can read actual error rates, correlate them with recent deployments, and suggest the specific commit that likely caused a spike.

This chapter covers MCP at the level of practical daily use — not the protocol specification, but the developer's experience of working with it. It explains how MCP connections work, which MCP servers are most valuable for different development workflows, and how to configure and use the five most important connections: database, GitHub, filesystem tools, web search, and communication tools (Slack and Notion). It also covers the skill of writing prompts that use multiple MCP connections simultaneously — orchestrating Claude Code across several live tools to answer questions or execute workflows that no single tool could address alone.

Chapter 12 later in the book shows a complete MCP-based automation in production — an email-to-task workflow that runs on a schedule. This chapter is the foundation for that: the concepts, the configuration patterns, and the mental model for thinking about what Claude Code can do when it is genuinely connected to your world rather than just informed about it.

One practical note before diving in: MCP servers are actively evolving. The servers and configuration details in this chapter reflect the state of the ecosystem as of mid-2026. The principles — how to think about MCP connections, how to write prompts that leverage them, how to combine multiple connections effectively — are durable. Specific configuration keys and server names may need to be verified against current documentation when you set them up.

What MCP actually is — beyond the jargon

The Model Context Protocol is an open standard, developed by Anthropic, that defines how an AI model can communicate with external tools and services. It is not a feature specific to Claude Code — it is a protocol that any AI application can implement, and it is designed to be composable: any number of MCP servers can be connected to a single model session simultaneously.

An MCP server is a small program that exposes a set of tools — named, typed functions that the model can call during a session. Each tool has a name, a description, and a parameter schema. When an MCP server is connected to a Claude Code session, Claude Code reads the tool list and understands what each tool does from its description. During the session, when a task would benefit from using one of those

tools, Claude Code calls it, receives the result, and incorporates the result into its response. You do not need to tell Claude Code when to use which tool — the model figures out which tools are relevant to the task at hand.

The configuration is straightforward. MCP servers are defined in a JSON file — typically .claude/mcp.json in your project directory, or ~/.claude/mcp.json for tools you want available in every project. Each entry specifies how to start the server (usually an npx command), and what environment variables it needs for authentication. Claude Code reads this file when a session starts and maintains connections to each configured server throughout the session.

mcp.json — basic structure

```
# .claude/mcp.json — minimal example
{
  "mcpServers": {
    "postgres": {
      "command": "npx",
      "args": ["-y", "@anthropic-ai/mcp-server-postgres"],
      "env": { "DATABASE_URL": "${DATABASE_URL}" }
    },
    "github": {
      "command": "npx",
      "args": ["-y", "@anthropic-ai/mcp-server-github"],
      "env": { "GITHUB_TOKEN": "${GITHUB_TOKEN}" }
    }
  }
}
# Environment variables are read from .env — never commit real values
```

The tools exposed by an MCP server are available to Claude Code exactly the way built-in capabilities are — the model just uses them when they are relevant. The difference from built-in capabilities is that MCP tools operate on real, live data from an external service, not on information embedded in the model's training data or provided in the context window.

The MCP ecosystem — which servers matter most for developers

As of mid-2026, dozens of MCP servers exist covering everything from cloud services to local development tools. The table below covers the servers most relevant for the kinds of development work this book addresses. The first-party servers (maintained by Anthropic) are the most reliable starting point; community servers vary in quality and maintenance status.

MCP SERVER	CONNECTS TO	MOST USEFUL FOR
@anthropic-ai/mcp-server-postgres	PostgreSQL database	Schema exploration, query writing, data analysis, identifying slow queries
@anthropic-ai/mcp-server-github	GitHub API	Reading issues, PRs, code search, branch management, release notes
@anthropic-ai/mcp-server-filesystem	Local file system	Reading/writing files outside the project directory, file search
@anthropic-ai/mcp-server-brave-search	Brave web search	Looking up documentation, checking library versions, current events
@anthropic-ai/mcp-server-slack	Slack workspace	Reading channel messages, searching history, posting summaries
@anthropic-ai/mcp-server-notion	Notion workspace	Creating pages, reading databases, managing tasks

tools, Claude Code calls it, receives the result, and incorporates the result into its response. You do not need to tell Claude Code when to use which tool — the model figures out which tools are relevant to the task at hand.

The configuration is straightforward. MCP servers are defined in a JSON file — typically .claude/mcp.json in your project directory, or ~/.claude/mcp.json for tools you want available in every project. Each entry specifies how to start the server (usually an npx command), and what environment variables it needs for authentication. Claude Code reads this file when a session starts and maintains connections to each configured server throughout the session.

mcp.json — basic structure

```
# .claude/mcp.json — minimal example
{
  "mcpServers": {
    "postgres": {
      "command": "npx",
      "args": ["-y", "@anthropic-ai/mcp-server-postgres"],
      "env": { "DATABASE_URL": "${DATABASE_URL}" }
    },
    "github": {
      "command": "npx",
      "args": ["-y", "@anthropic-ai/mcp-server-github"],
      "env": { "GITHUB_TOKEN": "${GITHUB_TOKEN}" }
    }
  }
}
# Environment variables are read from .env — never commit real values
```

The tools exposed by an MCP server are available to Claude Code exactly the way built-in capabilities are — the model just uses them when they are relevant. The difference from built-in capabilities is that MCP tools operate on real, live data from an external service, not on information embedded in the model's training data or provided in the context window.

The MCP ecosystem — which servers matter most for developers

As of mid-2026, dozens of MCP servers exist covering everything from cloud services to local development tools. The table below covers the servers most relevant for the kinds of development work this book addresses. The first-party servers (maintained by Anthropic) are the most reliable starting point; community servers vary in quality and maintenance status.

MCP SERVER	CONNECTS TO	MOST USEFUL FOR
@anthropic-ai/mcp-server-postgres	PostgreSQL database	Schema exploration, query writing, data analysis, identifying slow queries
@anthropic-ai/mcp-server-github	GitHub API	Reading issues, PRs, code search, branch management, release notes
@anthropic-ai/mcp-server-filesystem	Local file system	Reading/writing files outside the project directory, file search
@anthropic-ai/mcp-server-brave-search	Brave web search	Looking up documentation, checking library versions, current events
@anthropic-ai/mcp-server-slack	Slack workspace	Reading channel messages, searching history, posting summaries
@anthropic-ai/mcp-server-notion	Notion workspace	Creating pages, reading databases, managing tasks

@anthropic-ai/mcp-server-gmail	Gmail inbox	Reading emails, filtering, drafting responses
@anthropic-ai/mcp-server-linear	Linear issue tracker	Reading issues, creating tasks, updating status
@anthropic-ai/mcp-server-sentry	Sentry error monitoring	Reading error reports, correlating with code changes

When deciding which MCP servers to add to a project's .claude/mcp.json, the question is simple: which external services does working on this codebase require me to context-switch to most often? If you spend thirty minutes per week copying database schema information into prompts, the Postgres MCP eliminates that. If you frequently copy GitHub issue descriptions into Claude Code sessions, the GitHub MCP eliminates that. Install the MCPs that eliminate your most frequent manual context transfer — not all of them at once.

START WITH TWO MCPS, NOT TEN

More MCP connections do not automatically make Claude Code more effective. Each active connection adds tools to the session's context, which can dilute the model's focus for tasks that do not need those tools. Start with the two or three MCP connections that would eliminate the most manual context transfer from your current workflow. Add more as you identify specific needs — not because they sound useful in theory.

Connecting Claude to your database

The database MCP is the connection that changes the most about how Claude Code can help with backend development. Without it, every database-related prompt requires you to describe the schema, the data patterns, and the query context. With it, Claude Code can inspect the actual schema, query real data, run EXPLAIN on actual queries, and give you advice grounded in your specific database rather than a generalized mental model of what your database probably looks like.

The Postgres MCP exposes tools for listing tables, reading schema definitions, running SELECT queries, and running EXPLAIN ANALYZE on queries to understand their performance characteristics. It does not expose INSERT, UPDATE, or DELETE tools by default — the design principle is read access for analysis, write access only when you explicitly add it and understand the implications. This is the right default: you want Claude Code to understand your data, not to modify it autonomously.

PROMPT 1 — Database exploration and query optimization

Database exploration prompt

```
> Using the database connection, help me understand the
  performance issue with the projects listing page.

  Steps:
  1. List all tables in the database
  2. Show the schema for the projects, users, and project_members tables
  3. Run EXPLAIN ANALYZE on this query (the one the listing page uses):
     SELECT p.*, u.name as client_name,
            COUNT(pm.user_id) as member_count
     FROM projects p
     LEFT JOIN users u ON p.client_id = u.id
     LEFT JOIN project_members pm ON p.id = pm.project_id
     WHERE p.status = 'active'
     GROUP BY p.id, u.name
     ORDER BY p.created_at DESC
     LIMIT 20

  4. Identify the performance bottleneck
  5. Suggest the specific indexes that would help
  6. Show me the CREATE INDEX statements
```

The output from this kind of prompt is qualitatively different from anything Claude Code can produce without the database connection. It reads the actual execution plan — the real cost estimates, the real row counts, the real join strategies the query planner chose — and identifies the specific bottleneck in the actual query on the actual schema. The index suggestions are for the specific columns being filtered and joined in your codebase, not for a generalized 'users table.'

Schema-aware code generation

The database MCP enables a particularly powerful workflow: schema-aware code generation. When Claude Code has direct access to your database schema, generating accurate Drizzle schemas, SQLAlchemy models, or raw SQL queries becomes substantially more reliable because the model is reading the actual schema rather than inferring it from context.

Schema-aware Drizzle generation

```
> Read the complete schema from the database.
  Generate Drizzle ORM table definitions for all tables
  that do not already have a corresponding definition in
  src/db/schema.ts.

  For each new table:
  - Use the exact column names and types from the database
  - Infer TypeScript types from SQL types
    (uuid → uuid, text → text, boolean → boolean, etc.)
  - Add relations using Drizzle's relations API
  - Flag any columns where the naming convention differs from
    our camelCase TypeScript convention
```

Connecting to GitHub

The GitHub MCP turns Claude Code into a contributor to your issue tracker, not just your codebase. Instead of reading an issue number in a Slack message, switching to GitHub to read the full context, copying relevant details into a Claude Code session, and then switching back, you describe the task and Claude Code reads the issue directly.

The GitHub MCP tools cover the operations developers use most frequently: reading issue details and comments, searching code across the repository, reading pull request diffs and review comments, listing recent commits with their messages, and searching issues by label or milestone. It does not (by default) create or modify issues or PRs — again, the read-first principle preserves your control over what changes in your repository.

PROMPT 2 — Issue-to-fix workflow using GitHub MCP

GitHub issue-to-fix prompt

```
> Read GitHub issue #347. Understand the bug being reported.
  Then:
  1. Search the codebase for the code paths that handle
     the feature mentioned in the issue
  2. Identify the root cause of the bug
  3. Write the fix
  4. Write tests that would catch this bug and verify the fix
  5. Write a PR description explaining the change,
     referencing the issue number
```

GitHub MCP tool calls behind the scenes

```
# What the GitHub MCP does behind the scenes

# Tool: github.get_issue
```

```
# Result: full issue title, body, comments, labels, assignee

# Tool: github.search_code
# Query: the feature or function name mentioned in the issue
# Result: files and line numbers where relevant code lives

# Tool: github.get_file_contents
# Result: the actual code in the relevant files

# Claude Code then synthesizes: issue context + code context
# → root cause analysis → fix → tests → PR description
# All in one session, with no manual copy-pasting
```

Code search at scale

One of the most underrated capabilities the GitHub MCP unlocks is repository-wide code search. Asking 'where in this codebase is the payment retry logic implemented?' in a standard Claude Code session requires you to already know approximately where to look. With the GitHub MCP, Claude Code can search the entire repository for relevant function names, patterns, and comments — and bring the relevant code into the session automatically.

Repository-wide code search with GitHub MCP

```
> Search the codebase for all places where we handle
  payment failures or retries. I want to understand whether
  our retry logic is consistent across all payment flows.

  For each location found:
  - Show the retry strategy used (fixed delay, exponential backoff,
    max attempts)
  - Note any inconsistencies between different payment paths
  - Identify which approach should be standardized
```

Filesystem and web search tools

Two MCP connections deserve brief treatment because they extend Claude Code's reach in practical ways that do not require external service accounts: the filesystem MCP and a web search MCP.

Filesystem MCP — beyond the project directory

Claude Code already has access to the files in the current project directory. The filesystem MCP extends this to any path on your machine — useful for referencing documentation you have saved locally, reading configuration files outside the project, or working with scripts and tools that live in your home directory rather than the project repository.

Filesystem MCP — cross-directory reference

```
> Using the filesystem tool, read my ~/notes/api-contracts/
  directory and find any contract definitions relevant to
  the payment service. Then check whether our current
  implementation in src/services/payments.ts matches
  the contract specification.
```

□ FILESYSTEM MCP AND SENSITIVE FILES

The filesystem MCP can access any file your user account can read — including SSH keys, browser cookies, environment files containing production secrets, and other sensitive material. Configure it with a restricted path allowlist in your mcp.json rather than granting unrestricted access:

"args": ["-y", "@anthropic-ai/mcp-server-filesystem", "/Users/you/projects", "/Users/you/notes"]
List only the directories Claude Code should access, not the entire filesystem.

Web search MCP — live documentation

The web search MCP (Brave Search or similar) is most valuable for questions about recently released library versions, API changes, and documentation for tools that postdate Claude Code's training cutoff. When you are integrating a library that was released or significantly updated in the last year, the web search MCP lets Claude Code verify its knowledge against current documentation rather than relying on potentially outdated training data.

Web search MCP — library verification

```
> Before generating the Stripe webhook handler, use web search
  to verify the current recommended approach for signature
  verification in the Stripe Node.js SDK v16+. Check for
  any deprecations or changes from the v14 approach I
  have seen in older examples.
```

Building multi-tool MCP workflows

The most powerful use of MCP is not using a single connection but orchestrating Claude Code across multiple connections simultaneously. A session with the database MCP, the GitHub MCP, and web search active can answer questions and execute workflows that span all three — questions like 'what database queries are associated with the code changed in the last five pull requests, and are any of them missing indexes?'

Multi-tool workflows follow a natural structure: gather context from multiple sources simultaneously, synthesize the gathered information into analysis or a plan, then execute the output in the codebase. The gather phase is where multiple MCPs add the most value; the synthesize and execute phases use Claude Code's reasoning and code generation capabilities, which are not MCP-specific.

PROMPT 3 — Multi-tool workflow — feature from issue to implementation

Multi-tool implementation planning prompt

```
> I want to implement the feature described in GitHub issue #412.
  Use all available tools to give me a complete, grounded
  implementation plan before writing any code.

  Step 1 — Context gathering (use MCP tools):
    - GitHub: read issue #412 and all its comments
    - GitHub: search for any related issues or PRs
    - Database: read the schema for tables likely involved
    - Web search: check current best practices for the
      specific approach suggested in the issue

  Step 2 — Synthesis:
    - What exactly is being requested?
    - What does the current database schema support?
    - What changes to the schema (if any) are needed?
    - What are the files that need to change?

  Step 3 — Implementation plan:
    List every file to create or modify, in implementation order.
    Show the interface contracts (function signatures, API shapes)
    before writing any implementation code.

  Present the plan and wait for my approval before coding.
```

The value of this workflow is that the implementation plan is grounded in real, current information — the actual issue text and comments, the actual database schema, the actual best practices from current documentation — rather than in reconstructed context that was manually copied from those sources. The plan is also more likely to be correct the first time, because it accounts for constraints that might not have been obvious when the issue was described without the full codebase and database context.

The context-bridging prompt pattern

When working with multiple MCP connections, a useful pattern is the context-bridging prompt: explicitly telling Claude Code to gather information from one tool to inform its use of another. This makes the cross-tool reasoning explicit rather than leaving it implicit.

Context-bridging prompt patterns

```
# Context-bridging prompt patterns

# Bridge: GitHub issue → database schema
> Read issue #523. Identify the data entities it mentions.
  Then use the database connection to read the current schema
  for those entities and identify any gaps between what
  the issue requires and what the schema currently supports.

# Bridge: database slow query → GitHub history
> This query is running slowly (EXPLAIN output below).
  Search GitHub for recent commits that modified the files
  responsible for generating this query pattern.
  Determine whether a recent change introduced the performance
  regression and identify the commit if so.

# Bridge: Slack discussion → code change
> Read the last 20 messages in the #backend channel.
  Find any discussion of the caching behavior and understand
  what the team decided. Then look at the current caching
  implementation and determine whether it matches the decision.
```

Configuring MCP in your project CLAUDE.md

When MCP servers are configured for a project, the CLAUDE.md should document what each connection provides and how Claude Code should use it. This documentation serves two purposes: it reminds Claude Code to use the tools when they are relevant (the model is more likely to use a tool when it has been told explicitly that the tool is available and what it is for), and it helps new developers on the team understand the project's MCP setup.

CLAUDE.md MCP documentation section

```
## MCP connections (in .claude/mcp.json)

### Database (postgres)
Connected to the development PostgreSQL database.
Use for: schema exploration, query analysis, EXPLAIN plans.
Do NOT use for: INSERT/UPDATE/DELETE — read-only for analysis.

### GitHub (github)
Connected to github.com/yourorg/your-repo.
Use for: reading issues and PRs, code search, commit history.
When a prompt references an issue number, always read it first.
```

```
### Web search (brave-search)
Use for: verifying library documentation, checking for recent
API changes in libraries updated after mid-2025.
Always verify before using patterns from libraries released
or significantly updated after 2024.

### Slash commands that use MCP
/review-pr   — uses GitHub to read the PR diff
/analyze-db  — uses Postgres for schema and query analysis
/fix-issue N — uses GitHub to read issue N before implementing
```

Project: A code intelligence assistant using MCP

This project builds a Claude Code workflow that uses three MCP connections simultaneously — database, GitHub, and web search — to answer a common developer question that previously required significant manual research: 'Is the code in this repository consistent with what the database actually has, and are there any obvious performance or correctness issues I should know about?'

The workflow is not a deployed application. It is a documented set of prompts — a slash command you add to your project — that produces a structured report on the current state of the data layer. Run it once a sprint or when taking over an unfamiliar codebase. The output is actionable: specific tables with missing indexes, specific queries with performance issues, specific places where the ORM schema does not match the database schema.

PROMPT 4 — Data layer health report

Slash command: /data-health

```
# .claude/commands/data-health.md
# Usage: /data-health

Produce a data layer health report for this codebase.
Use all available MCP tools. Work through these checks:

CHECK 1 — Schema consistency:
  Read the database schema for all tables.
  Compare with the ORM definitions in src/db/schema.ts.
  List any discrepancies: columns in DB not in ORM,
  columns in ORM not in DB, type mismatches.

CHECK 2 — Index coverage:
  List all foreign key columns across all tables.
  Check which have corresponding indexes.
  List the foreign keys that are missing indexes.
  These are N+1 query hazards.

CHECK 3 — Query pattern analysis:
  Search the codebase for raw SQL queries and ORM queries.
  Identify queries that filter on columns without indexes.
  Flag queries that use SELECT * (should select specific columns).

CHECK 4 — Recent regressions:
  Using GitHub, read the last 10 commits that touched
  database-related files (migrations, schema, queries).
  Flag any that removed indexes, changed column types,
  or added queries that filter on unindexed columns.

Format the report as:
## Data Layer Health Report — [date]
### Critical (fix before next deploy)
```

```
### Important (fix this sprint)
### Low priority (track in backlog)
### All clear (things that look good)
```

This slash command embodies the core value proposition of multi-tool MCP workflows: work that would previously require a developer to spend two to three hours manually checking schemas, reading migration history, and grepping for query patterns is now a five-minute automated report. The report is grounded in real, current data — not a static documentation audit, but a live check against the actual state of the database and the actual state of the code.

Run this command when joining a new project, when taking ownership of a module you have not worked in recently, or when investigating unexplained performance degradation. The report will not catch everything — it is not a replacement for a dedicated DBA review or a comprehensive performance audit — but it catches the most common categories of data layer debt in minutes rather than hours.

✓ WHAT MCP GIVES YOU THAT PROMPTING ALONE CANNOT

Context that is always current: database schemas change, issues get updated, documentation evolves. MCP connects Claude Code to the live state of these things, not to your memory of what they were when you last described them in a prompt.

Reduced context transfer friction: the single biggest tax on AI-assisted development is the manual work of getting information from external sources into the prompt. MCP eliminates that tax for the sources it connects to.

Cross-tool reasoning: the most valuable questions in software development cross system boundaries. MCP makes it possible to ask those questions and get answers grounded in real data from multiple real systems simultaneously.

CHAPTER 9 SUMMARY

MCP turns Claude Code from a coding assistant into a development environment that can see and interact with the real systems your code depends on. The Model Context Protocol is an open standard that defines how AI models connect to external tools; MCP servers expose tools that the model calls during a session without you directing each call explicitly.

The highest-value MCP connections for most developers are the database (schema exploration, query analysis), GitHub (issue reading, code search, PR context), and web search (documentation verification for recent library changes). Multi-tool workflows orchestrate Claude Code across several connections simultaneously, enabling questions and workflows that span system boundaries.

Configure MCP connections in .claude/mcp.json, document them in CLAUDE.md, and create slash commands for the multi-tool workflows you run repeatedly. The /data-health command in this chapter's project is a template for the category: a routine, MCP-powered audit that surfaces actionable findings in minutes rather than hours.

Chapter 10 builds on this MCP foundation to cover CI/CD and DevOps integration — how to use Claude Code to generate Dockerfiles, GitHub Actions workflows, and Terraform configurations, and how to deploy them safely.